geometry.py

```
def classify_lines(line1, line2, line3):
                A = line1
                B = line2
                C = line3

                def det(a, b):
            return a['A'] * b['B'] - a['B'] * b['A']

                def det3(a, b, c):
        return (a['A'] * (b['B'] * c['C'] - b['C'] * c['B'])
              - a['B'] * (b['A'] * c['C'] - b['C'] * c['A'])
             + a['C'] * (b['A'] * c['B'] - b['B'] * c['A']))

                def is_zero(x, eps=1e-9):
                     return abs(x) < eps

                # Pairwise 2x2 determinants
                     d_AB = det(A, B)
                     d_AC = det(A, C)
                     d_BC = det(B, C)

                  mix_det = det3(A, B, C)

                def intersection(l1, l2):
                       d = det(l1, l2)
                       if is_zero(d):
                           return None
           x = (l1['B'] * l2['C'] - l1['C'] * l2['B']) / d
           y = (l1['C'] * l2['A'] - l1['A'] * l2['C']) / d
                       return (x, y)

                # Class 1: All lines coincide
      if is_zero(d_AB) and is_zero(line1['A']*line2['C'] -
                  line2['A']*line1['C']) \
    and is_zero(line1['A']*line3['B'] - line3['A']*line1['B']) \
    and is_zero(line1['A']*line3['C'] - line3['A']*line1['C']):
                           return 1, []

       # Class 2: All lines parallel (some could coincide)
            if is_zero(d_AB) and is_zero(d_AC) and \
not is_zero((line1['A']*line2['C'] - line2['A']*line1['C'])**2 +
                                \
```

```
                (line1['A']*line3['C'] - line3['A']*line1['C'])**2):
                    return 2, []

                # Class 3: All lines meet at one point
    if (not is_zero(d_AB) or not is_zero(d_AC) or not is_zero(d_BC)) and
                        is_zero(mix_det):
                    point = intersection(A, B)
                return 3, [point] if point else []

        # Class 4: Two intersections (two lines meet, one parallel or
                            coincide)
     num_nonparallel = sum(not is_zero(d) for d in (d_AB, d_AC, d_BC))
                    if num_nonparallel == 2:
                            points = []
                        if not is_zero(d_AB):
                    points.append(intersection(A, B))
                        if not is_zero(d_AC):
                    points.append(intersection(A, C))
                        if not is_zero(d_BC):
                    points.append(intersection(B, C))
                           return 4, points

        # Class 5: Three pairwise intersections (all lines different)
              if num_nonparallel == 3 and not is_zero(mix_det):
                            points = [
                        intersection(A, B),
                        intersection(A, C),
                         intersection(B, C)
                                ]
                        return 5, points

                        return 0, []
```

app.py

```
                    from geometry import *

                    def try_parse_int(s):
                            try:
                              try:
                            qq = float(s)
                          if '.' in s in s:
                print(f"Помилка: {s} не є ЦІЛИМ числом, спробуйте ще
                            раз")
```

```
                return None
            except ValueError:
                pass

            res = int(s)

        if not (-127 <= res <= 127):
    print(f"Число {res} поза діапазоном [-127; 127], спробуйте
            ще раз")
                return None
            return res
        except ValueError:
print(f"Помилка: {s} не є ЦІЛИМ числом, спробуйте ще раз")
            return None

    def validate_a_b(a, b):
        if a**2 + b**2 != 0:
            return True
    print("Помилка: A^2 + B^2 != 0, спробуйте ще раз")
            return False

    def try_input_A_B():
            while True:
                while True:
        A = try_parse_int(input("a (для Ax+By+C=0): "))
                if A is not None:
                    break

                while True:
        B = try_parse_int(input("b (для Ax+By+C=0): "))
                if B is not None:
                    break
            if validate_a_b(A, B):
                return A, B

    def try_parse_b(b, index):
        b = try_parse_int(b)
        if b is not None:
            if b == 0:
    print(f"b{index} не може бути 0, спробуйте ще раз")
                return None
            return b
```

```
                    def try_input_b(index):
                            while True:
            b = input(f"b{index} (для y=k{index}x+b{index}): ")
                        b = try_parse_b(b, index)
                            if b is not None:
                                return b

                            def main():
                        N = 127  # Згідно з варіантом
                  print("Введіть параметри трьох прямих:")

                            # Введення даних
                                try:
                            a, b = try_input_A_B()
                  c = try_parse_int(input("c (для Ax+By+C=0): "))

                            while c is None:
                   c = try_parse_int(input("c (для Ax+By+C=0): "))

                k1 = try_parse_int(input("k1 (для y=k1x+b1): "))
                            while k1 is None:
                   k1 = try_parse_int(input("k1 (для y=k1x+b1): "))
                            b1 = try_input_b(1)

                k2 = try_parse_int(input("k2 (для y=k2x+b2): "))
                            while k2 is None:
                   k2 = try_parse_int(input("k2 (для y=k2x+b2): "))
                            b2 = try_input_b(2)

                            except ValueError as e:
                              print(f"Помилка: {e}")
                                    return

                        # Convert input to line format
                        line1 = {'A': a, 'B': b, 'C': c}
line2 = {'A': -k1, 'B': 1, 'C': -b1}  # Convert y = k1x + b1 to -k1x
                                + y - b1 = 0
line3 = {'A': -k2, 'B': 1, 'C': -b2}  # Convert y = k2x + b2 to -k2x
                                + y - b2 = 0

                            # Classify the lines
```

```
    classification, points = classify_lines(line1, line2, line3)

    # Output results based on classification
    print()
    if classification == 1:
        print("Прямі співпадають")
    elif classification == 2:
        print("Прямі паралельні або деякі співпадають")
    elif classification == 3:
        print("Прямі перетинаються в одній точці")
        print(f"Точка перетину: ({points[0][0]:.5f},
              {points[0][1]:.5f})")
    elif classification == 4:
        print("Прямі перетинаються в двох точках")
        for i, point in enumerate(points, 1):
            print(f"Точка {i}: ({point[0]:.5f}, {point[1]:.5f})")
    elif classification == 5:
        print("Прямі перетинаються в трьох різних точках")
        for i, point in enumerate(points, 1):
            print(f"Точка {i}: ({point[0]:.5f}, {point[1]:.5f})")
    else:
        print("Невідомий випадок взаємного розташування прямих")

if __name__ == "__main__":
    while True:
        print()
        main()
```

tests.py

```
import pytest
from unittest.mock import patch
from app import try_parse_int, validate_a_b, try_parse_b, try_input_b
# adjust import
from geometry import classify_lines

# ----------------- try_parse_int -----------------

@pytest.mark.parametrize("input_value, expected", [
    ("42", 42),
    ("-127", -127),
    ("127", 127),
    ("128", None),
    ("-128", None),
```

```
                            ("abc", None),
                                ])
        def test_try_parse_int(input_value, expected, capsys):
                    result = try_parse_int(input_value)
                      captured = capsys.readouterr()

                        assert result == expected
                          if expected is None:
                    assert "спробуйте ще раз" in captured.out

            # ----------------- validate_a_b -----------------

               @pytest.mark.parametrize("a, b, expected", [
                                (1, 0, True),
                                (0, 1, True),
                                (3, 4, True),
                               (0, 0, False),
                                    ])
              def test_validate_a_b(a, b, expected, capsys):
                        result = validate_a_b(a, b)
                      captured = capsys.readouterr()

                        assert result == expected
                             if not expected:
                    assert "спробуйте ще раз" in captured.out

            # ----------------- try_parse_b -----------------
           @pytest.mark.parametrize("input_value, expected", [
                                ("42.5", None),
                               ("-127.0", None),
                              ("127.999", None),
                                ("0.0", None),
                                ("1.23", None),
                                    ])
    def test_try_parse_int_float_error(input_value, expected, capsys):
                    result = try_parse_int(input_value)
                      captured = capsys.readouterr()

                        assert result == expected
                assert "не є ЦІЛИМ числом" in captured.out

                def test_try_parse_b_valid(monkeypatch):
            monkeypatch.setattr('builtins.input', lambda _: "25")
```

```
                    result = try_input_b(1)
                    assert result == 25

    def test_try_parse_b_invalid_then_valid(monkeypatch):
                inputs = iter(["abc", "100"])
monkeypatch.setattr('builtins.input', lambda _: next(inputs))
                    result = try_input_b(2)
                    assert result == 100

        # ----------------- try_input_b -----------------

            def test_try_input_b_valid(monkeypatch):
        monkeypatch.setattr('builtins.input', lambda _: "45")
                assert try_input_b(1) == 45

    def test_try_input_b_invalid_then_valid(monkeypatch):
                inputs = iter(["-200", "50"])
monkeypatch.setattr('builtins.input', lambda _: next(inputs))
                assert try_input_b(2) == 50

            def make_lines(a, b, c, k1, b1, k2, b2):
                line1 = {'A': a, 'B': b, 'C': c}
                line2 = {'A': -k1, 'B': 1, 'C': -b1}
                line3 = {'A': -k2, 'B': 1, 'C': -b2}
                    return line1, line2, line3

        # ----------------- classify_lines -----------------

    @pytest.mark.parametrize("inputs, expected_class", [
                        # Class 1 (coincide)
                ((-127, -1, 0, -127, 0, -127, 0), 1),
                 ((127, -1, 0, 127, 0, 127, 0), 1),
                    ((1, -1, 0, 1, 0, 1, 0), 1),
                  ((127, -127, 0, 1, 0, 1, 0), 1),
                 ((-127, -127, 0, -1, 0, -1, 0), 1),

                        # Class 2 (parallel)
               ((127, -1, 127, 127, 126, 127, 125), 2),
            ((-127, -1, -127, -127, -126, -127, -125), 2),
                    ((1, -1, 0, 1, 2, 1, 3), 2),
              ((-127, -127, -127, -1, 2, -1, -127), 2),
             ((-127, -1, -127, -127, -126, -127, 2), 2),
```

```
        # Class 3 (intersect at one point)
        ((127, -127, 127, 126, 0, 126, 0), 3),
        ((-127, -127, -127, -126, 0, -126, 0), 3),
        ((1, 1, 0, 8, 0, 4, 0), 3),
        ((-127, -127, -127, -2, -2, 127, 127), 3),
        ((-127, -127, -127, -126, -126, -1, -1), 3),
        ((127, 127, 127, 126, 126, 1, 1), 3),

        # Class 4 (intersect at two points)
        ((127, 127, 127, 126, 126, 126, 125), 4),
        ((-127, -127, -127, -126, -126, -126, -125), 4),
        ((1, 1, 0, 2, 1, 2, 2), 4),
        ((-127, -127, -127, -1, 2, 127, 127), 4),
        ((-127, -127, -127, -126, -124, -1, 2), 4),
        ((127, 127, 127, 126, 124, -1, 2), 4),

        # Class 5 (intersect at three points)
        ((127, 127, 0, 126, 126, 125, 126), 5),
        ((-127, -127, 0, -126, -126, -125, -126), 5),
        ((1, 1, 0, 1, 0, 2, -2), 5),
        ((-127, -127, -127, 1, 0, 126, 126), 5),
        ((-127, -127, -127, -127, -126, 1, 0), 5),
        ((127, 127, 127, 126, 125, 1, 0), 5),
    ])


def test_classify_lines(inputs, expected_class):
    a, b, c, k1, b1, k2, b2 = inputs
    line1, line2, line3 = make_lines(a, b, c, k1, b1, k2, b2)

    classification, points = classify_lines(line1, line2, line3)

    assert classification == expected_class
```